

One-Dimensional Arrays

Primitive · Object · CRUD · Sort · Filter · Process

Chapter Contents

01 Primitive Arrays

`int[]` · declare · index · defaults

02 Array Operations

`sum` · `min/max` · `insert` · `delete`

03 Object Arrays · OOP

`Employee[]` · `refs` · `encapsulation`

04 CRUD · Manager

`add` · `find` · `update` · `remove`

05 Sort · Filter · Comparator

`Arrays.sort` · `Comparator`

06 Stream · Looking Ahead

`lambda` · `parallel` · `Collections` next

Learning Path

1

Primitive Array

declare · access · traverse

2

Array Operations

CRUD patterns on int[]

3

Object Array · OOP

references · Department

4

CRUD · Sort · Filter

Manager API on Employee[]

5

Arrays · Comparator

sort · binarySearch · comparing

6

Stream · Parallel

lambda · parallel stream · bridge

01 · ARRAY FUNDAMENTALS

Array Fundamentals

Concept · Declaration · Index · Default values · Access & update

What Is an Array?

An array is a fixed-size, ordered collection of elements of the same type. Each element is stored in a contiguous slot and addressed by an index.

Fixed size

Length is set when the array is created and cannot grow

Same type

All elements share one type — `int[]`, `String[]`, `Employee[]`

Indexed access

Fast random access via index starting at 0

Reference type

The array variable holds a reference to the array object

Array as a Row of Slots

```
int[] scores = { 8, 7, 9, 6, 10 };
```



Index starts at 0 · Last index is length - 1
Length of this array is 5 · scores[0] is 8 · scores[4] is 10

Index and length

Index range

Valid indices: 0 .. length - 1

array.length

Public final field — not a method (no parentheses)

Out of bounds

scores[5] on length 5 → `ArrayIndexOutOfBoundsException`

Loop pattern

```
for (int i = 0; i < scores.length; i++)
```

Default Values

When you create an array with `new Type[n]`, every slot is filled with a default value.

Element type	Default value	Example
byte, short, int, long	0	<code>new int[3] → {0,0,0}</code>
float, double	0.0	<code>new double[2]</code>
char	<code>'\u0000'</code>	<code>new char[2]</code>
boolean	false	<code>new boolean[2]</code>
Reference types	null	<code>new String[3]</code>

02 · WORKING WITH 1D ARRAYS

Working with 1D Arrays

Declare · Assign · Access · for · for-each · Input / Output

Declaring an Array

```
// Type[] name; or Type name[];  
int[] scores;  
double[] prices;  
String[] names;  
  
// Preferred style in Java: Type[] name  
int[] scores; // scores is null until assigned  
  
// Create the array object  
scores = new int[5]; // length fixed at 5  
  
// One-liner  
int[] scores = new int[5];
```

Assigning Values

```
// At creation — initializer list  
int[] scores = { 8, 7, 9, 6, 10 };
```

```
// Slot by slot after creation  
int[] scores = new int[5];  
scores[0] = 8;  
scores[1] = 7;  
scores[2] = 9;  
scores[3] = 6;  
scores[4] = 10;
```

```
// From a loop (e.g. fill with i + 1)  
for (int i = 0; i < scores.length; i++) {  
    scores[i] = i + 1;  
}
```

Accessing Elements

```
int[] scores = { 8, 7, 9, 6, 10 };
```

```
int first = scores[0];           // 8  
int third = scores[2];           // 9  
int last  = scores[scores.length - 1]; // 10
```

```
scores[1] = 10; // update
```

```
// Invalid index → runtime exception  
// scores[-1]; scores[5];  
// ArrayIndexOutOfBoundsException
```

Traversing with for

Use an index for loop when you need the position, or when you write into the array.

```
int[] scores = { 8, 7, 9, 6, 10 };

// Read each element
for (int i = 0; i < scores.length; i++) {
    System.out.println "[" + i + "] = " + scores[i];
}

// Update each element
for (int i = 0; i < scores.length; i++) {
    scores[i] = scores[i] * 2;
}
```

Traversing with for-each

Enhanced for is shorter when you only read values. You do not get the index, and you cannot assign a new value into the array slot through the loop variable.

```
int[] scores = { 8, 7, 9, 6, 10 };
```

```
for (int score : scores) {  
    System.out.println(score);  
}
```

```
// score = 0; does NOT change the array  
// Use indexed for when you need to write
```

for vs for-each

for (index)

- Need the index
- Need to update slots
- Need to walk backward
- Need only part of the range

```
for (int i = 0; i < a.length; i++)  
    a[i] = ...;
```

for-each

- Read-only traversal
- Cleaner syntax
- No index available
- Cannot replace elements

```
for (int x : a)  
    System.out.println(x);
```

Input / Output of an Array

```
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] scores = new int[n];

// Input
for (int i = 0; i < scores.length; i++) {
    scores[i] = sc.nextInt();
}

// Output
for (int i = 0; i < scores.length; i++) {
    System.out.print(scores[i] + " ");
}
System.out.println();

// Or: System.out.println(Arrays.toString(scores));
```

Key Takeaways — Working with 1D Arrays

- 01 Declare with `Type[] name`; create with `new` or `{ ... }`
- 02 Assign at creation or slot-by-slot with `name[i] = value`
- 03 Indexed for: read, write, and use the index
- 04 for-each: simple read-only traversal
- 05 Input/output usually uses an indexed for + `Scanner` / `print`

03 · BASIC ARRAY OPERATIONS

Basic Array Operations

Search · Sum · Average · Min / Max · Count · Reverse · Swap

Search

```
// Linear search — return index or -1
int indexOf(int[] a, int key) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == key) return i;
    }
    return -1; // not found
}
```

```
int[] scores = { 8, 7, 9, 6, 10 };
int pos = indexOf(scores, 9); // 2
boolean found = pos >= 0;
```

Sum & Average

```
int sum(int[] a) {  
    int total = 0;  
    for (int value : a) total += value;  
    return total;  
}  
  
double average(int[] a) {  
    if (a.length == 0) return 0;  
    return (double) sum(a) / a.length;  
}  
  
int[] scores = { 8, 7, 9, 6, 10 };  
sum(scores);    // 40  
average(scores); // 8.0
```

Min & Max

// Start from a[0] — works with negatives too

```
int min(int[] a) {
```

```
    int m = a[0]; // NOT 0
```

```
    for (int i = 1; i < a.length; i++) {
```

```
        if (a[i] < m) m = a[i];
```

```
    }
```

```
    return m;
```

```
}
```

```
int max(int[] a) {
```

```
    int m = a[0]; // NOT 0
```

```
    for (int i = 1; i < a.length; i++) {
```

```
        if (a[i] > m) m = a[i];
```

```
    }
```

```
    return m;
```

```
}
```

```
int[] a = { -5, 3, -2, 9, -10 };
```

```
min(a); // -10    max(a); // 9
```

```
// Wrong: int max = 0; → fails when all values are negative
```

Count

```
// How many times does key appear?  
int count(int[] a, int key) {  
    int n = 0;  
    for (int value : a) {  
        if (value == key) n++;  
    }  
    return n;  
}  
  
// Count elements that satisfy a condition  
int countPass(int[] a, int threshold) {  
    int n = 0;  
    for (int value : a) {  
        if (value >= threshold) n++;  
    }  
    return n;  
}
```

Reverse

// Reverse in place — two pointers

```
void reverse(int[] a) {
```

```
    int left = 0;
```

```
    int right = a.length - 1;
```

```
    while (left < right) {
```

```
        int temp = a[left];
```

```
        a[left] = a[right];
```

```
        a[right] = temp;
```

```
        left++;
```

```
        right--;
```

```
    }
```

```
}
```

```
int[] a = { 1, 2, 3, 4, 5 };
```

```
reverse(a); // { 5, 4, 3, 2, 1 }
```

Swap

// Swap two elements by index

```
void swap(int[] a, int i, int j) {  
    int temp = a[i];  
    a[i] = a[j];  
    a[j] = temp;  
}
```

```
int[] a = { 8, 7, 9, 6, 10 };  
swap(a, 0, 4); // { 10, 7, 9, 6, 8 }
```

// Building block for sort / reverse / shuffle
// Always use a temporary variable

Key Takeaways — Basic Operations

- 01 Search: scan with a loop · return index or -1
- 02 Sum / average: accumulate · watch empty array for average
- 03 Min / max: start from `a[0]` · compare the rest
- 04 Count: tally matches or conditions
- 05 Reverse: two pointers + swap toward the center
- 06 Swap: temp variable · core of many algorithms

04 · INSERT · DELETE · UPDATE

Insert · Delete · Update

Arrays have fixed size — insert and delete need shifting (or a new array)

Fixed Size — The Key Constraint

Update

Easy — overwrite a slot: $a[i] = \text{value}$

Insert

Need a free slot (or a larger array) + shift elements right

Delete

Shift elements left to close the gap (logical size shrinks)

Logical size

Track how many slots are “in use” when $\text{capacity} > \text{count}$

Update

```
// Overwrite the value at index i
void update(int[] a, int index, int value) {
    if (index < 0 || index >= a.length) {
        throw new IndexOutOfBoundsException();
    }
    a[index] = value;
}
```

```
int[] scores = { 8, 7, 9, 6, 10 };
update(scores, 1, 10); // { 8, 10, 9, 6, 10 }
```

// $O(1)$ — no shifting required

Insert — Shift Right

To insert at index i , move $a[i] \dots a[n-1]$ one step to the right, then write the new value at i . Need spare capacity (logical size < length).

Before · size = 4 · capacity = 5

[8 | 7 | 9 | 6 | _] insert 10 at index 1

After · size = 5

[8 | 10 | 7 | 9 | 6]

Insert — Code

```
// a has capacity a.length; size = number of used slots
// Returns new size, or -1 if full / bad index
int insert(int[] a, int size, int index, int value) {
    if (size >= a.length) return -1; // full
    if (index < 0 || index > size) return -1;
    for (int i = size; i > index; i--) {
        a[i] = a[i - 1]; // shift right
    }
    a[index] = value;
    return size + 1;
}

int[] a = new int[5];
int size = 0;
size = insert(a, size, 0, 8); // size becomes 1
```

Delete — Shift Left

To delete at index i , move $a[i+1] \dots a[\text{size}-1]$ one step to the left. Logical size decreases by 1. The last slot may still hold an old value — ignore it via size.

Before · size = 5

[8 | 10 | 7 | 9 | 6] delete index 1

After · size = 4

[8 | 7 | 9 | 6 | (6)] last slot ignored

Delete — Code

```
int delete(int[] a, int size, int index) {  
    if (index < 0 || index >= size) return size;  
    for (int i = index; i < size - 1; i++) {  
        a[i] = a[i + 1]; // shift left  
    }  
    return size - 1;  
}
```

```
int[] a = { 8, 10, 7, 9, 6 };  
int size = 5;  
size = delete(a, size, 1); // logical: 8,7,9,6
```

```
// O(n) because of shifting
```

Update vs Insert vs Delete

Operation	Idea	Cost
Update	Overwrite $a[i]$	$O(1)$
Insert	Shift right, write value	$O(n)$
Delete	Shift left, $size--$	$O(n)$

Key Takeaways — Insert · Delete · Update

- 01 Update is direct assignment — $O(1)$
- 02 Insert needs free capacity and shifts elements right
- 03 Delete shifts elements left and decreases logical size
- 04 Track size separately when capacity > number of used elements
- 05 For frequent insert/delete, prefer List (ArrayList) later

05 · SORT · SEARCH · MERGE

Sort · Binary Search · Duplicates · Merge

Ordered data · fast lookup · clean sets · combine arrays

Sorting an Array

Sorting rearranges elements in order (ascending or descending). After sorting, binary search becomes possible.

```
// Built-in — preferred in practice
import java.util.Arrays;
int[] a = { 8, 3, 5, 1, 9 };
Arrays.sort(a); // { 1, 3, 5, 8, 9 }
```

```
// Descending (for objects / boxed types later)
// Or write a simple selection / bubble sort for learning
```

Selection Sort (for learning)

```
void selectionSort(int[] a) {  
    for (int i = 0; i < a.length - 1; i++) {  
        int minIdx = i;  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[j] < a[minIdx]) minIdx = j;  
        }  
        // swap a[i] and a[minIdx]  
        int temp = a[i];  
        a[i] = a[minIdx];  
        a[minIdx] = temp;  
    }  
}  
  
// O(n2) — fine for small n and teaching
```

Binary Search — Idea

Works only on a sorted array. Compare the key with the middle element; discard half of the range each step. Much faster than linear search for large n .

Prerequisite

Array must be sorted (ascending here)

Each step

$\text{mid} = (\text{left} + \text{right}) / 2$ · compare $a[\text{mid}]$ with key

Complexity

$O(\log n)$ vs $O(n)$ for linear search

Result

Index of key, or -1 if not found

Binary Search — Code

```
int binarySearch(int[] a, int key) {  
    int left = 0;  
    int right = a.length - 1;  
    while (left <= right) {  
        int mid = left + (right - left) / 2; // avoid overflow  
        if (a[mid] == key) return mid;  
        if (a[mid] < key) left = mid + 1;  
        else right = mid - 1;  
    }  
    return -1;  
}
```

```
int[] a = { 1, 3, 5, 8, 9 }; // sorted  
binarySearch(a, 8); // 3  
// Arrays.binarySearch(a, 8); built-in
```

Linear Search vs Binary Search

Linear

- Any order
- Scan from start
- $O(n)$
- Simple
- Good for small arrays
or unsorted data

Binary

- Must be sorted
- Halve the range
- $O(\log n)$
- Faster for large n
- Pay the cost of
sorting first if needed

Remove Duplicates

On a sorted array, duplicates sit next to each other — easy to compact in place.

```
// Sorted input · returns new logical size
int removeDuplicates(int[] a, int size) {
    if (size <= 1) return size;
    int w = 1; // write index
    for (int r = 1; r < size; r++) {
        if (a[r] != a[w - 1]) {
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
// {1,1,2,2,2,3} → {1,2,3,...} size 3
```

Merge Two Arrays

```
// Concatenate a and b into a new array
int[] merge(int[] a, int[] b) {
    int[] c = new int[a.length + b.length];
    for (int i = 0; i < a.length; i++) c[i] = a[i];
    for (int i = 0; i < b.length; i++) {
        c[a.length + i] = b[i];
    }
    return c;
}
```

```
// Sorted merge (both a and b already sorted)
// Two pointers → one sorted result (merge step of mergesort)
```

Merge Two Sorted Arrays

```
int[] mergeSorted(int[] a, int[] b) {  
    int[] c = new int[a.length + b.length];  
    int i = 0, j = 0, k = 0;  
    while (i < a.length && j < b.length) {  
        if (a[i] <= b[j]) c[k++] = a[i++];  
        else c[k++] = b[j++];  
    }  
    while (i < a.length) c[k++] = a[i++];  
    while (j < b.length) c[k++] = b[j++];  
    return c;  
}  
  
mergeSorted({1,4,7}, {2,3,8}); // {1,2,3,4,7,8}
```

Key Takeaways — Sort · Search · Merge

- 01 `Arrays.sort` for real code · write a sort to learn
- 02 Binary search needs sorted data · $O(\log n)$
- 03 Remove duplicates is easy after sorting (adjacent equals)
- 04 Merge: copy concatenate, or two-pointer sorted merge
- 05 Choose linear vs binary based on order and size

06 · ARRAY AS A CLASS

IntArray — OOP Demo

data[] + count · operations as methods · one case study

Why Wrap the Array in a Class?

Encapsulation

Hide `data[]` and `count` — callers use methods, not raw indices only

Logical size

`count` tracks how many slots are in use; capacity is `data.length`

Cohesive API

`insert`, `delete`, `search`, `sum...` live with the data they work on

OOP practice

Same style as `Employee` — state + behavior in one type

IntArray — Structure

```
public class IntArray {  
    private int[] data; // capacity = data.length  
    private int count; // logical size (0 .. capacity)  
  
    public IntArray(int capacity) {  
        data = new int[capacity];  
        count = 0;  
    }  
  
    public int size() { return count; }  
    public int capacity() { return data.length; }  
    public boolean isEmpty() { return count == 0; }  
    public boolean isFull() { return count == data.length; }  
}
```

IntArray Object — Mental Model

IntArray

count = 4

data (capacity = 6)

[	8		7		9		6		_		_	]
	0		1		2		3		4		5	

Used: indices 0 .. count-1

Free: count .. capacity-1

add · set · get

```
public void add(int value) { // append
    if (isFull()) throw new IllegalStateException("full");
    data[count] = value;
    count++;
}
public void set(int index, int value) { // update
    checkIndex(index);
    data[index] = value;
}
public int get(int index) {
    checkIndex(index);
    return data[index];
}
private void checkIndex(int i) {
    if (i < 0 || i >= count) throw new IndexOutOfBoundsException();
}
```

insert · remove — Methods

```
public void insert(int index, int value) {  
    if (isFull()) throw new IllegalStateException("full");  
    if (index < 0 || index > count) throw new IndexOutOfBoundsException();  
    for (int i = count; i > index; i--) data[i] = data[i - 1];  
    data[index] = value;  
    count++;  
}
```

```
public void remove(int index) {  
    checkIndex(index);  
    for (int i = index; i < count - 1; i++) data[i] = data[i + 1];  
    count--;  
}
```

Query Methods — sum · min · max · indexOf

```
public int sum() {  
    int s = 0;  
    for (int i = 0; i < count; i++) s += data[i];  
    return s;  
}  
  
public int min() { // starts at data[0] — OK with negatives  
    int m = data[0];  
    for (int i = 1; i < count; i++) if (data[i] < m) m = data[i];  
    return m;  
}  
  
public int indexOf(int key) {  
    for (int i = 0; i < count; i++) if (data[i] == key) return i;  
    return -1;  
}
```

Using IntArray

```
IntArray scores = new IntArray(10);
```

```
scores.add(8);  
scores.add(7);  
scores.add(9);  
scores.insert(1, 10); // 8, 10, 7, 9  
scores.set(2, 6);    // 8, 10, 6, 9  
scores.remove(1);    // 8, 6, 9  
  
System.out.println(scores.size()); // 3  
System.out.println(scores.sum());  // 23  
System.out.println(scores.indexOf(6)); // 1
```

When Full — Grow the Capacity

If `count == data.length`, allocate a larger array, copy elements, then continue the insert/add. Typical growth: double the capacity (like `ArrayList`).

```
private void ensureCapacity() {  
    if (count < data.length) return; // still have room  
    int newCap = data.length * 2;  
    int[] bigger = new int[newCap];  
    for (int i = 0; i < count; i++) bigger[i] = data[i];  
    data = bigger; // replace reference  
}
```

```
public void add(int value) {  
    ensureCapacity(); // grow if full  
    data[count++] = value;  
}
```

Remove — Shift Left

```
public void remove(int index) {  
    checkIndex(index);  
    for (int i = index; i < count - 1; i++) {  
        data[i] = data[i + 1]; // shift left  
    }  
    count--;  
    // int: leftover in data[count] is ignored — no null  
}
```

Before: [8 | 10 | 7 | 9 | _] count = 4

`remove(1);`

After: [8 | 7 | 9 | 9 | _] count = 3

used 0..2 ignored →

After Remove — int vs Object

int[] (primitive)

- No null for int
- Shift left, then count--
- Old value may remain in data[count]
- Harmless — loops use 0 .. count-1 only

```
// optional clear (not required)
```

```
// data[count] = 0;
```

Object[] (e.g. Employee[])

- Shift left, then count--
- Set last used slot to null
- Helps garbage collection
- Avoids lingering references

```
data[count] = null; // important
```

remove on Object[] — null the tail

```
// Inside a class holding Employee[] data
public void remove(int index) {
    checkIndex(index);
    for (int i = index; i < count - 1; i++) {
        data[i] = data[i + 1];
    }
    count--;
    data[count] = null; // drop reference for GC
}

// Default for new Employee[n]: every slot is already null
// After remove: clear the abandoned slot explicitly
```

Remove All Elements — `clear()`

Removing every element does not need shifting one by one. Reset the logical size. Capacity can stay the same for reuse.

`int[]` / `IntArray`

`count = 0` — old values ignored; no null exists for int

`Object[]`

Null every used slot, then `count = 0` — release references for GC

Capacity

Usually keep `data.length`; next `add()` reuses the array

Cost

$O(1)$ for int · $O(n)$ for `Object[]` if you null out slots

clear() on IntArray

```
public void clear() {  
    count = 0; // logical size empty  
    // data[] still allocated — ready for reuse  
}
```

```
IntArray a = new IntArray(8);  
a.add(8); a.add(7); a.add(9);  
a.clear();  
a.size();    // 0  
a.isEmpty(); // true  
a.add(1);    // reuses same data[]  
  
// Optional: zero out (rarely needed for int)  
// for (int i = 0; i < data.length; i++) data[i] = 0;
```

clear() on Object[]

```
// Employee[] data; int count;  
public void clear() {  
    for (int i = 0; i < count; i++) {  
        data[i] = null; // drop references for GC  
    }  
    count = 0;  
}
```

// Why null?
// Without it, data[0].. still point to Employee objects
// GC cannot reclaim them until the array is replaced

// ArrayList.clear() does the same idea

Remove All Occurrences of a Value

Example: remove every 2 from {1, 2, 3, 4, 5, 2, 3, 2} → {1, 3, 4, 5, 3} (count shrinks).

Not remove(index) in a loop

Shifting from the front repeatedly is $O(n^2)$ and easy to get wrong

Better: one pass

Copy keepers forward with a write pointer · then set count

Order preserved

Relative order of remaining elements stays the same

Idea — Write Pointer

Scan with read index r .

If $\text{data}[r]$ is NOT the key, write it at w and advance w .

At the end, $\text{count} = w$.

key = 2

Before: [1 | 2 | 3 | 4 | 5 | 2 | 3 | 2] $\text{count} = 8$

After: [1 | 3 | 4 | 5 | 3 | ...] $\text{count} = 5$

Kept in order · removed all 2s · one $O(n)$ pass

removeAll(key) — Code

```
/** Remove every element equal to key. Returns how many were removed. */
public int removeAll(int key) {
    int w = 0; // write index
    for (int r = 0; r < count; r++) {
        if (data[r] != key) {
            data[w] = data[r];
            w++;
        }
    }
    int removed = count - w;
    count = w;
    return removed;
}

// {1,2,3,4,5,2,3,2} · removeAll(2) → {1,3,4,5,3} · returns 3
```

Why Not `remove(index)` in a Loop?

X Fragile / slow

```
for (int i = 0; i < count; i++) {  
    if (get(i) == key)  
        remove(i);  
}
```

- Index skips after shift
- $O(n^2)$ shifts
- Easy off-by-one bugs

✓ `removeAll(key)`

One forward pass
Write keepers only
`count = w`

- $O(n)$
- Stable order
- No index confusion

Key Takeaways — Array as a Class

- 01 `data[] = capacity · count = logical size`
- 02 All operations are methods — OOP style
- 03 `add / insert / remove` maintain count
- 04 `get / set / indexOf / sum / min` use `0 .. count-1`
- 05 Same ideas as raw arrays — clearer structure for OOP courses

07 · TWO-DIMENSIONAL ARRAYS

Two-Dimensional Arrays

Declare · Access · Traverse · Matrix view

What Is a 2D Array?

A 2D array is an array of arrays — rows of elements. In Java it is often used as a matrix: `a[row][col]`.

Rows and columns

First index = row · second index = column

Rectangular

Every row has the same length (common case)

Jagged possible

In Java, rows can have different lengths if created separately

Use cases

Tables · grids · matrices · game boards · pixel data

Matrix View — `a[row][col]`

1 [0][0]	2 [0][1]	3 [0][2]	4 [0][3]
5 [1][0]	6 [1][1]	7 [1][2]	8 [1][3]
9 [2][0]	10 [2][1]	11 [2][2]	12 [2][3]

`a[0][0] = 1` · `a[1][2] = 7` · `a[2][3] = 12` · 3 rows × 4 columns

Declaration & Creation

```
// Declare  
int[][] a;  
  
// Create rectangular 3 rows × 4 columns  
a = new int[3][4]; // all zeros by default  
  
// One-liner  
int[][] a = new int[3][4];  
  
// Create and initialize  
int[][] a = {  
    { 1, 2, 3, 4 },  
    { 5, 6, 7, 8 },  
    { 9, 10, 11, 12 }  
};
```

Accessing Elements

```
int[][] a = {  
    { 1, 2, 3, 4 },  
    { 5, 6, 7, 8 },  
    { 9, 10, 11, 12 }  
};
```

```
int x = a[1][2]; // 7 — row 1, column 2  
a[0][3] = 40;    // update
```

```
int rows = a.length;    // 3  
int cols = a[0].length; // 4 (if rectangular)
```

```
// a[row][col] · row in 0..rows-1 · col in 0..cols-1
```

Traversing — Nested for

```
// Row by row (most common)
for (int r = 0; r < a.length; r++) {
    for (int c = 0; c < a[r].length; c++) {
        System.out.print(a[r][c] + " ");
    }
    System.out.println();
}
```

```
// for-each over rows, then elements
for (int[] row : a) {
    for (int value : row) {
        System.out.print(value + " ");
    }
    System.out.println();
}
```

Matrix — Common Tasks

Sum of all elements

Nested loop · accumulate $a[r][c]$

Row sum / column sum

Fix r and vary c · or fix c and vary r

Diagonal

Main: $a[i][i]$ · secondary: $a[i][n-1-i]$ (square matrix)

Transpose

$b[c][r] = a[r][c]$ (or swap in place if square)

Search

Nested scan · or smarter methods if sorted

Example — Sum & Row Sum

```
int sumAll(int[][] a) {  
    int s = 0;  
    for (int r = 0; r < a.length; r++)  
        for (int c = 0; c < a[r].length; c++)  
            s += a[r][c];  
    return s;  
}  
  
int rowSum(int[][] a, int r) {  
    int s = 0;  
    for (int c = 0; c < a[r].length; c++) s += a[r][c];  
    return s;  
}
```

Key Takeaways — 2D Arrays

- 01 `int[][] a = new int[rows][cols]` or initializer lists
- 02 Access with `a[row][col]` · bounds for each dimension
- 03 `a.length` = number of rows · `a[r].length` = columns in that row
- 04 Nested for (or for-each) to traverse
- 05 Think of a matrix: sum, row/col, diagonal, transpose...

09 · ARRAYS IN OOP

From `int[]` to `Employee[]`

Primitive arrays · Object arrays · references in slots

So Far — Arrays of Primitives

`int[]`, `double[]`, ... store values directly in each slot. Good for numbers and simple data — not enough when each item has several fields and behavior.

`int[] scores`

`[ 8 | 7 | 9 | 6 ]`

Each slot holds a number.

No name, no id, no methods.

Need more?

An employee has:

- id
- name
- maybe pay()

One int cannot hold that.

Transition — Parallel Arrays → Objects

Without classes, people often keep several arrays in parallel. That breaks easily. Objects fix the design.

X Parallel arrays

```
String[] ids = {"E01","E02"};  
String[] names = {"An","Binh"};  
  
// ids[i] must match names[i]  
// Easy to desync on insert/delete
```

✓ Array of objects

```
Employee[] staff;
```

Each element is one person:

id + name together.

Operations use methods.

Employee — the Element Type

```
public class Employee {  
    private String employeeId;  
    private String name;  
  
    public Employee(String employeeId, String name) {  
        this.employeeId = employeeId;  
        this.name = name;  
    }  
  
    public String getEmployeeId() { return employeeId; }  
    public String getName() { return name; }  
  
    @Override  
    public String toString() {  
        return employeeId + " - " + name;  
    }  
}
```

What Does Employee[] Store?

Slots hold references, not the objects themselves. `new Employee[n]` fills every slot with null until you assign real objects.

```
Employee[] staff = new Employee[3];
```

```
[ null | null | null ]
```

```
staff[0] = new Employee("E01", "An");
```

```
[ (E01,An) | null | null ] ← reference in slot 0
```

Calling `staff[0].getName()` uses the object through the reference.

Declare · Create · Fill

```
Employee[] staff = new Employee[3]; // all null
```

```
staff[0] = new Employee("E01", "An");  
staff[1] = new Employee("E02", "Binh");  
staff[2] = new Employee("E03", "Chi");
```

// Or initializer

```
Employee[] staff = {  
    new Employee("E01", "An"),  
    new Employee("E02", "Binh"),  
    new Employee("E03", "Chi")  
};
```

Traverse & Use Objects

```
// Indexed for
for (int i = 0; i < staff.length; i++) {
    if (staff[i] != null) {
        System.out.println(staff[i].getName());
    }
}

// for-each
for (Employee e : staff) {
    if (e != null) System.out.println(e); // toString()
}

// Null check matters — empty slots are null
```

Search — Find by employeeId

```
Employee findByEmployeeId(Employee[] staff, String employeeId) {  
    for (Employee e : staff) {  
        if (e != null && e.getEmployeeId().equals(employeeId)) {  
            return e;  
        }  
    }  
    return null; // not found  
}
```

```
Employee e = findByEmployeeId(staff, "E02");  
if (e != null) System.out.println(e.getName());
```

int[] vs Employee[]

int[]

- Slot = value
- Default 0
- `a[i] = 10`
- Compare with `==`
- No methods on element
- Simple numbers only

Employee[]

- Slot = reference
- Default null
- `a[i] = new Employee(...)`
- Compare id with `equals`
- Call methods on element
- Rich domain objects

Key Takeaways — Arrays in OOP

- 01 Primitive arrays store values · object arrays store references
- 02 Prefer `Employee[]` over parallel `String[]` id / name arrays
- 03 `new Employee[n]` → all null until you assign objects
- 04 Always consider null when traversing or searching
- 05 Elements carry both data and behavior (`getId`, `toString`, ...)

Reference and Object

`staff[i]` does not contain the `Employee`. It holds a reference (address) to an object on the heap.

Variable / slot

```
Employee e;  
staff[0];
```

Type: reference

May be null or point to one object

Object

```
new Employee("E01","An")
```

Lives on the heap

id + name inside

Reached only via a reference

Two References, One Object

```
Employee[] staff = new Employee[2];  
staff[0] = new Employee("E01", "An");  
Employee e = staff[0]; // same object
```

e and staff[0] point to ONE Employee
// Changing through e is visible via staff[0]

```
staff[1] = staff[0]; // two slots, one object  
// Not a copy of fields — shared reference
```

null in Object Arrays

Default

`new Employee[5]` → every slot is null

Meaning

"No object here yet" — empty seat in the array

Danger

`staff[i].getName()` when `staff[i]` is null → `NullPointerException`

Practice

Check null before use · or keep a logical count of filled slots

Clear slot

`staff[i] = null` releases the reference (helps GC)

Array + Constructor

Creating the array is not the same as creating the elements. You construct each Employee explicitly.

```
Employee[] staff = new Employee[3]; // array only
// No Employee constructed yet

staff[0] = new Employee("E01", "An"); // constructor runs
staff[1] = new Employee("E02", "Binh");
staff[2] = new Employee("E03", "Chi");

// Initializer list = create array + construct each element
Employee[] staff = {
    new Employee("E01", "An"),
    new Employee("E02", "Binh")
};
```

Array + Encapsulation

Do not expose the raw `Employee[]` if you can avoid it. Hide it inside a class and offer controlled methods.

```
public class Department {  
    private Employee[] staff; // hidden  
    private int count;  
  
    public Department(int capacity) {  
        staff = new Employee[capacity];  
        count = 0;  
    }  
  
    public void add(Employee e) { /* ... */ }  
    public Employee findById(String employeeId) { /* ... */ }  
    // no public Employee[] getStaff() that leaks the array  
}
```

Why Hide the Array?

Invariants

count stays consistent · no null holes in the used range

Safety

Callers cannot overwrite `staff[i]` arbitrarily

API

`add` / `remove` / `findByEmployeeId` describe intent better than raw indices

Change later

You can switch to `ArrayList` without breaking callers

Key Takeaways — Reference · null · Design

- 01 Array slot holds a reference · object lives on the heap
- 02 Several slots can share one object (aliasing)
- 03 null = empty slot · check before calling methods
- 04 new Employee[n] creates the array only — construct each element
- 05 Encapsulate Employee[] inside a class (Department) when possible

10 · MANAGING OBJECTS WITH ARRAY

Add · Expand · Remove

Department holds Employee[] · count · controlled operations

Department — Holder of Employee[]

```
public class Department {  
    private Employee[] staff;  
    private int count;  
  
    public Department(int capacity) {  
        staff = new Employee[capacity];  
        count = 0;  
    }  
  
    public int size() { return count; }  
    public boolean isFull() { return count == staff.length; }  
}
```

add — Throw When Full

```
public void add(Employee e) {  
    if (e == null) throw new IllegalArgumentException("null");  
    if (isFull()) {  
        throw new IllegalStateException("Department is full");  
    }  
    staff[count] = e;  
    count++;  
}
```

```
Department d = new Department(2);  
d.add(new Employee("E01", "An"));  
d.add(new Employee("E02", "Binh"));  
d.add(new Employee("E03", "Chi")); // throws
```

add — Expand When Full

```
// Constructor already set capacity >= 1 → length never 0
```

```
private void ensureCapacity() {  
    if (count < staff.length) return;  
    int newCap = staff.length * 2; // double  
    Employee[] bigger = new Employee[newCap];  
    for (int i = 0; i < count; i++) bigger[i] = staff[i];  
    staff = bigger;  
}  
  
public void add(Employee e) {  
    if (e == null) throw new IllegalArgumentException("null");  
    ensureCapacity(); // grow if full  
    staff[count++] = e;  
}  
  
// Optional: require capacity >= 1 in constructor
```

Full Policy — Exception vs Expand

Throw Exception

- Fixed capacity by design
- Caller must handle full case
- Simple, predictable memory
- Good for teaching limits

`IllegalStateException`

Expand

- Grows like `ArrayList`
- Double capacity often
- Amortized $O(1)$ add
- More flexible for apps

`ensureCapacity()`

remove(index) — Shift + null

```
public void remove(int index) {  
    if (index < 0 || index >= count)  
        throw new IndexOutOfBoundsException();  
    for (int i = index; i < count - 1; i++) {  
        staff[i] = staff[i + 1]; // shift left  
    }  
    count--;  
    staff[count] = null; // drop reference for GC  
}  
  
// Object[] — always null the abandoned slot
```

removeAllByName — All Matches

Remove every Employee whose name equals the given name. One pass with a write pointer (same idea as removeAll on int[]).

```
public int removeAllByName(String name) {  
    int w = 0;  
    for (int r = 0; r < count; r++) {  
        if (!staff[r].getName().equals(name)) {  
            staff[w++] = staff[r]; // keep  
        }  
    }  
    int removed = count - w;  
    for (int i = w; i < count; i++) staff[i] = null;  
    count = w;  
    return removed;  
}
```

Example — removeAllByName

```
Department d = new Department(10);  
d.add(new Employee("E01", "An"));  
d.add(new Employee("E02", "Binh"));  
d.add(new Employee("E03", "An"));  
d.add(new Employee("E04", "Chi"));
```

```
int n = d.removeAllByName("An"); // n = 2  
// Remaining: Binh, Chi  
d.size(); // 2
```

Key Takeaways — Manage Objects

- 01 add: reject null · full → exception or ensureCapacity()
- 02 remove(index): shift left · staff[count] = null
- 03 removeAllByName: one write-pointer pass · null leftover slots
- 04 count is the single source of truth for logical size
- 05 Same patterns as IntArray — now with references + GC

Update Object

```
// Replace the Employee at index (or update fields)
public void update(int index, Employee e) {
    if (index < 0 || index >= count)
        throw new IndexOutOfBoundsException();
    if (e == null) throw new IllegalArgumentException("null");
    staff[index] = e; // replace reference
}
```

```
// Or update by employeeId — find then replace
public boolean updateByEmployeeId(String employeeId, Employee e) {
    int i = indexOfEmployeeId(employeeId);
    if (i < 0) return false;
    staff[i] = e;
    return true;
}
```

Find Object — by employeeId

```
public int indexOfEmployeeId(String employeeId) {  
    for (int i = 0; i < count; i++) {  
        if (staff[i].getEmployeeId().equals(employeeId)) return i;  
    }  
    return -1;  
}
```

```
public Employee findByEmployeeId(String employeeId) {  
    int i = indexOfEmployeeId(employeeId);  
    return i >= 0 ? staff[i] : null;  
}
```

```
Employee e = d.findByEmployeeId("E02");  
if (e != null) System.out.println(e.getName());
```

toString — Not display()

// Domain class describes state — caller prints

@Override

```
public String toString() {  
    if (count == 0) return "(empty)";  
    StringBuilder sb = new StringBuilder();  
    for (int i = 0; i < count; i++) {  
        if (i > 0) sb.append(", ");  
        sb.append(staff[i]); // Employee.toString()  
    }  
    return sb.toString();  
}
```

System.out.println(department); // uses toString()

// Avoid System.out inside Department

Find Many — by name

Return all employees matching a name. Two-pass or dynamic count: first count matches, then fill a result array.

```
public Employee[] findByName(String name) {  
    int n = 0;  
    for (int i = 0; i < count; i++) {  
        if (staff[i].getName().equals(name)) n++;  
    }  
    Employee[] result = new Employee[n];  
    int w = 0;  
    for (int i = 0; i < count; i++) {  
        if (staff[i].getName().equals(name))  
            result[w++] = staff[i];  
    }  
    return result; // length 0 if none  
}
```

Example — findByName

```
Department d = new Department(10);  
d.add(new Employee("E01", "An"));  
d.add(new Employee("E02", "Binh"));  
d.add(new Employee("E03", "An"));
```

```
Employee[] list = d.findByName("An");  
for (Employee e : list) {  
    System.out.println(e); // E01 - An , E03 - An  
}
```

```
Employee[] none = d.findByName("Zoe"); // length 0
```

toString — Not display()

```
// Domain class returns text — caller prints
@Override
public String toString() {
    if (count == 0) return "(empty)";
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < count; i++) {
        if (i > 0) sb.append(", ");
        sb.append(staff[i]); // Employee.toString()
    }
    return sb.toString();
}
```

```
System.out.println(department); // uses toString()
// Do not put System.out inside Department
```

Sort Objects — by name or id

Sorting rearranges references in `staff[0..count-1]`. Compare String fields with `compareTo` (or `Comparator` later).

```
/** Sort by name ascending — selection sort on 0..count-1 */
public void sortByName() {
    for (int i = 0; i < count - 1; i++) {
        int min = i;
        for (int j = i + 1; j < count; j++) {
            if (staff[j].getName().compareTo(staff[min].getName()) < 0)
                min = j;
        }
        Employee tmp = staff[i];
        staff[i] = staff[min];
        staff[min] = tmp;
    }
}
```

sortByEmployeeId — Same Pattern

```
public void sortByEmployeeId() {  
    for (int i = 0; i < count - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < count; j++) {  
            if (staff[j].getEmployeeId().compareTo(staff[min].getEmployeeId()) < 0)  
                min = j;  
        }  
        Employee tmp = staff[i];  
        staff[i] = staff[min];  
        staff[min] = tmp;  
    }  
}
```

```
// Later: Arrays.sort(staff, 0, count, Comparator.comparing(Employee::getName));
```

Filter Objects

Filter returns a new array of matching employees — original Department is unchanged (unlike remove).

findByName

Already a filter — all with a given name

General filter

Any condition: name starts with, id prefix, ...

Result

Employee[] sized to the number of matches

filterByNamePrefix

```
public Employee[] filterByNamePrefix(String prefix) {  
    int n = 0;  
    for (int i = 0; i < count; i++) {  
        if (staff[i].getName().startsWith(prefix)) n++;  
    }  
    Employee[] result = new Employee[n];  
    int w = 0;  
    for (int i = 0; i < count; i++) {  
        if (staff[i].getName().startsWith(prefix))  
            result[w++] = staff[i];  
    }  
    return result;  
}
```

// filterByNamePrefix("A") → all names starting with A

Sort vs Filter

Sort

- Changes order in place
- Same elements, new order
- count unchanged
- `sortByName` / `sortByEmployeeId`
- Uses `compareTo`

Filter

- Returns a new array
- Subset of elements
- Original unchanged
- `findByName` / `prefix`
- Uses `condition` / `equals`

Key Takeaways — Sort · Filter

- 01 Sort: rearrange references in `0..count-1` (in place)
- 02 Compare fields with `compareTo` — not `<` on `String`
- 03 Filter: new `Employee[]` of matches — original stays
- 04 Same two-pass idea as `findByName` for building the result
- 05 Later: `Comparator` + `Arrays.sort` / `Streams`

12 · LIMITS & LOOKING AHEAD

Beyond a Fixed Array

size vs length · resize · dynamic array · generic array

Limits of a Plain Array

Fixed capacity

length is set at creation — cannot grow without a new array

Manual shifting

insert / delete in the middle is $O(n)$ and easy to get wrong

No built-in size

length $\neq$ "how many used" — you track count yourself

Type is rigid

`Employee[]` is only `Employee` — generics help reuse structure

Raw work repeats

Every project reimplements `ensureCapacity`, `removeAll`, ...

size vs length

length

- Field of the array object
- Capacity — how many slots exist
- Fixed after new Type[n]
- `staff.length`
- Never changes unless you replace the array

size / count

- Logical number of elements
- $0 \leq \text{count} \leq \text{length}$
- You maintain it in a class
- `department.size()`
- Grows/shrinks on add/remove

Resize — The Only Way to “Grow”

// Arrays do not resize in place

1. Allocate a larger array
2. Copy existing elements
3. Point the field to the new array

```
Employee[] bigger = new Employee[staff.length * 2];  
for (int i = 0; i < count; i++) bigger[i] = staff[i];  
staff = bigger; // old array becomes garbage if unused
```

// Cost: $O(n)$ copy when growing

// Strategy: grow by $\times 2 \rightarrow$ amortized $O(1)$ add

Building a Dynamic Array (Idea)

You already did this: `IntArray` and `Department`. That is a hand-built dynamic array — fixed array + count + grow.

`data[]` + `count`

Capacity vs logical size

`ensureCapacity()`

Resize when full (×2)

`add` / `remove`

Update count · shift when needed

API

Callers never touch the raw array

Generic Array — One Structure, Many Types

IntArray only holds int. With generics you write the structure once and reuse it for Employee, String, ...

```
public class DynamicArray<E> {  
    private E[] data;  
    private int count;  
  
    @SuppressWarnings("unchecked")  
    public DynamicArray(int capacity) {  
        data = (E[]) new Object[capacity]; // common pattern  
        count = 0;  
    }  
  
    public void add(E e) { ensureCapacity(); data[count++] = e; }  
    // remove, get, size ... same as before  
}
```

```
DynamicArray<Employee> staff = new DynamicArray<>(10);
```

Looking Ahead — Collections Framework

You built the ideas by hand. The next chapter introduces the standard library types that implement them — without redoing `ensureCapacity` every time.

What you learned here

Array · count · grow · OOP element types · polymorphic lists

What comes next

List, ArrayList, ... — ready-made dynamic arrays and more

This chapter stops here

We do not enter Collections API in detail yet

Key Takeaways

- 01 `length = capacity · size/count = how many in use`
- 02 `Grow only by allocating a new array and copying`
- 03 `Dynamic array = array + count + ensureCapacity + API`
- 04 `Generics reuse the same structure for many element types`
- 05 `Collections Framework builds on these ideas next`

13 · `java.util.Arrays`

The `Arrays` Utility Class

`sort` · `binarySearch` · `fill` · `copyOf` · `equals` · `toString` · ...

What Is `java.util.Arrays`?

A utility class of static methods for common array tasks. Prefer these over hand-written loops when they fit.

Static methods

Call `Arrays.sort(a)` — no instance

Primitives & objects

Overloads for `int[]`, `Object[]`, ...

Not a collection

Works on arrays only — still fixed length

Import

```
import java.util.Arrays;
```

toString · equals · deepEquals

```
int[] a = { 1, 2, 3 };  
System.out.println(a);           // [I@hash — not useful  
System.out.println(Arrays.toString(a)); // [1, 2, 3]
```

```
int[] b = { 1, 2, 3 };  
a == b;           // false (different objects)  
Arrays.equals(a, b); // true — same length & elements
```

```
int[][] m1 = { {1,2}, {3,4} };  
int[][] m2 = { {1,2}, {3,4} };  
Arrays.equals(m1, m2); // false — compares row refs  
Arrays.deepEquals(m1, m2); // true
```

sort · fill · parallelSort

```
int[] a = { 8, 3, 5, 1, 9 };
```

```
Arrays.sort(a);
```

```
Arrays.toString(a); // [1, 3, 5, 8, 9]
```

```
Arrays.sort(a, 1, 4); // sort only index 1..3
```

```
int[] b = new int[5];
```

```
Arrays.fill(b, 7); // [7, 7, 7, 7, 7]
```

```
Arrays.fill(b, 1, 4, 0); // [7, 0, 0, 0, 7]
```

```
// Arrays.parallelSort(a); large arrays — parallel
```

binarySearch — Sorted Only

```
int[] a = { 1, 3, 5, 8, 9 }; // must be sorted
int i = Arrays.binarySearch(a, 8); // 3
int j = Arrays.binarySearch(a, 4); // negative → not found

// If missing: result = -(insertionPoint) - 1
// insertionPoint = where key would go

if (j < 0) {
    int insertAt = -j - 1;
}

// Unsorted array → undefined result — sort first
```

copyOf · copyOfRange · arraycopy

```
int[] a = { 1, 2, 3, 4, 5 };
```

```
int[] b = Arrays.copyOf(a, a.length);    // full copy
```

```
int[] c = Arrays.copyOf(a, 8);           // grow, extra = 0
```

```
int[] d = Arrays.copyOfRange(a, 1, 4);    // [2, 3, 4]
```

```
// System.arraycopy — low-level, fast
```

```
int[] dest = new int[5];
```

```
System.arraycopy(a, 0, dest, 0, a.length);
```

```
// Useful inside ensureCapacity()
```

Sorting Objects

```
Employee[] staff = { ... };
```

```
// Natural order if Employee implements Comparable  
Arrays.sort(staff);
```

```
// Or Comparator (by name)  
Arrays.sort(staff, (e1, e2) ->  
    e1.getName().compareTo(e2.getName()));
```

```
// Sort only the used range in a Department  
Arrays.sort(staff, 0, count, Comparator.comparing(Employee::getEmployeeId));
```

Quick Reference

`toString / deepToString`

Readable print

`equals / deepEquals`

Content equality

`sort / parallelSort`

In-place order

`binarySearch`

$O(\log n)$ on sorted data

`fill`

Set all / range to a value

`copyOf / copyOfRange`

New array, optional grow

`arraycopy`

Fast block copy

Key Takeaways — Arrays Class

- 01 Prefer `Arrays.*` for `sort`, `search`, `copy`, `print`, `equals`
- 02 `binarySearch` requires a sorted array
- 03 `toString` / `equals` beat default `Object` behavior on arrays
- 04 `copyOf` is the clean way to grow capacity
- 05 Still an array — fixed length until you replace the reference

14 · COMPARATOR

Sorting with Comparator

External order · `Arrays.sort(array, comparator)`

Two Ways to Sort Objects

Way 1 — Comparable

- Order lives inside the class
- implements `Comparable<T>`
- `compareTo` defines "natural order"
- `Arrays.sort(staff)`
- One natural order only

Way 2 — Comparator

- Order lives outside the class
- Pass Comparator to sort
- Many orders: by name, by `employeeId`...
- `Arrays.sort(staff, cmp)`
- Class stays unchanged

Why Start with Comparator (Way 2)?

Employee may not be yours

You cannot always edit the class to add `compareTo`

Multiple orders

By name today, by `employeeId` tomorrow — without changing `Employee`

Clear separation

Data model vs sorting policy

Same `Arrays.sort` API

```
Arrays.sort(array, comparator)
```

Comparator<T> — The Contract

```
public interface Comparator<T> {  
    int compare(T a, T b);  
}
```

// Return:

// < 0 if a should come before b

// = 0 if a and b are equal for this order

// > 0 if a should come after b

```
import java.util.Comparator;
```

```
import java.util.Arrays;
```

Demo — sort by name (anonymous class)

```
Employee[] staff = {  
    new Employee("E03", "Chi"),  
    new Employee("E01", "An"),  
    new Employee("E02", "Binh")  
};
```

```
Arrays.sort(staff, new Comparator<Employee>() {  
    @Override  
    public int compare(Employee a, Employee b) {  
        return a.getName().compareTo(b.getName());  
    }  
});
```

```
// Order: An, Binh, Chi
```

Demo — sort by name (lambda)

```
// Same order — shorter syntax
Arrays.sort(staff, (a, b) ->
    a.getName().compareTo(b.getName()));

// By employeeId
Arrays.sort(staff, (a, b) ->
    a.getEmployeeId().compareTo(b.getEmployeeId()));

// Descending by name
Arrays.sort(staff, (a, b) ->
    b.getName().compareTo(a.getName()));
```

Comparator.comparing — Clean API

```
import java.util.Comparator;

Arrays.sort(staff,
    Comparator.comparing(Employee::getName));

Arrays.sort(staff,
    Comparator.comparing(Employee::getEmployeeId));

// Reverse
Arrays.sort(staff,
    Comparator.comparing(Employee::getName).reversed());

// Only the used range (Department)
Arrays.sort(staff, 0, count,
    Comparator.comparing(Employee::getName));
```

Reusable Comparator Class

```
public class EmployeeNameComparator  
    implements Comparator<Employee> {  
    @Override  
    public int compare(Employee a, Employee b) {  
        return a.getName().compareTo(b.getName());  
    }  
}
```

```
Arrays.sort(staff, new EmployeeNameComparator());
```

```
// Reuse in many places · easy to test
```

Key Takeaways — Comparator

- 01 Way 2: `Arrays.sort(array, comparator)` — order outside the class
- 02 `compare` returns `<0 / 0 / >0` like `compareTo`
- 03 Anonymous class · lambda · `Comparator.comparing`
- 04 Many policies: by name, by `employeeId`, ascending / descending
- 05 Comparable (Way 1) = one natural order inside the type

15 · LAMBDA · STREAM API

Modern Array Processing

Lambda · Stream · filter · map · forEach — on Employee[]

Lambda — Already Used with sort

A lambda is a short form of a single-method interface (like Comparator). Stream operations use the same idea.

```
// Comparator as lambda (from previous part)
Arrays.sort(staff, (a, b) ->
    a.getName().compareTo(b.getName()));

// Shape: (parameters) -> expression
// Or:    (parameters) -> { statements; }
```

From Employee[] to Stream

```
import java.util.Arrays;
import java.util.stream.Stream;

Employee[] staff = {
    new FullTimeEmployee("E01", "An", 15_000_000),
    new PartTimeEmployee("E02", "Binh", 100_000, 80),
    new FullTimeEmployee("E03", "Chi", 12_000_000),
    new PartTimeEmployee("E04", "An", 120_000, 40)
};

Stream<Employee> stream = Arrays.stream(staff);
// or: Stream.of(staff[0], staff[1], ...)
```

Stream Pipeline — Idea

1 · Source

`Arrays.stream(staff)` — the array

2 · Intermediate

`filter · map · sorted · distinct` — return a new stream

3 · Terminal

`forEach · collect · count · sum` — produce a result

forEach — Print All

```
// Classic
```

```
for (Employee e : staff) {  
    System.out.println(e);  
}
```

```
// Stream
```

```
Arrays.stream(staff)  
    .forEach(e -> System.out.println(e));
```

```
// Method reference
```

```
Arrays.stream(staff)  
    .forEach(System.out::println);
```

filter — Keep Matching Employees

```
// Names equal to "An"
Arrays.stream(staff)
    .filter(e -> e.getName().equals("An"))
    .forEach(System.out::println);

// Only full-time
Arrays.stream(staff)
    .filter(e -> e instanceof FullTimeEmployee)
    .forEach(System.out::println);

// pay() > 10_000_000
Arrays.stream(staff)
    .filter(e -> e.pay() > 10_000_000)
    .forEach(System.out::println);
```

map — Transform Each Element

```
// Employee → name (Stream<String>)
```

```
Arrays.stream(staff)
```

```
    .map(Employee::getName)
```

```
    .forEach(System.out::println);
```

```
// Employee → pay (DoubleStream)
```

```
double total = Arrays.stream(staff)
```

```
    .mapToDouble(Employee::pay)
```

```
    .sum();
```

```
System.out.println(total); // sum of all pay()
```

sorted · toArray

```
import java.util.Comparator;

Employee[] byName = Arrays.stream(staff)
    .sorted(Comparator.comparing(Employee::getName))
    .toArray(Employee[]::new); // back to array

// Filter then sort then array
Employee[] rich = Arrays.stream(staff)
    .filter(e -> e.pay() > 10_000_000)
    .sorted(Comparator.comparing(Employee::getName))
    .toArray(Employee[]::new);
```

Full Pipeline Demo

```
// Print names of part-timers, sorted, distinct
Arrays.stream(staff)
    .filter(e -> e instanceof PartTimeEmployee)
    .map(Employee::getName)
    .distinct()
    .sorted()
    .forEach(System.out::println);

// count how many full-time
long n = Arrays.stream(staff)
    .filter(e -> e instanceof FullTimeEmployee)
    .count();
```

Stream vs Classic Loop

Classic for

- Explicit index / for-each
- Mutable accumulators
- Full control
- Verbose for filter+map
- Easy to debug step-by-step

Stream

- Declarative pipeline
- filter / map / sorted
- Less boilerplate
- toArray when you need an array again
- Great for queries on data

Key Takeaways — Lambda · Stream

- 01 `Arrays.stream(staff)` turns `Employee[]` into a `Stream`
- 02 Lambda powers `filter`, `map`, `sorted`, `forEach`
- 03 Intermediate ops chain; terminal ops produce results
- 04 `mapToDouble(Employee::pay).sum()` for totals
- 05 `toArray(Employee[]::new)` brings data back to an array

Parallel Stream — Idea

A parallel stream splits the data and processes chunks on multiple threads (ForkJoinPool). Same pipeline API — different execution.

Sequential

One thread walks the stream end to end

Parallel

Work split across cores — can be faster for large data

Same operations

filter · map · sum ... look identical in code

Not always faster

Overhead for small arrays · order may change

Creating a Parallel Stream

```
// From array — parallel from the start  
Arrays.stream(staff).parallel()  
    .mapToDouble(Employee::pay)  
    .sum();
```

```
// Or switch an existing stream  
Arrays.stream(staff)  
    .parallel() // sequential → parallel  
    .filter(e -> e.pay() > 10_000_000)  
    .count();
```

```
// Back to sequential if needed  
// .sequential()
```

Demo — Parallel totalPay

```
Employee[] staff = { /* many employees */ };
```

```
// Sequential
```

```
double s1 = Arrays.stream(staff)  
    .mapToDouble(Employee::pay)  
    .sum();
```

```
// Parallel — same result for associative ops
```

```
double s2 = Arrays.stream(staff)  
    .parallel()  
    .mapToDouble(Employee::pay)  
    .sum();
```

```
// s1 == s2 for sum
```

Order Is Not Guaranteed

```
// forEach on parallel stream — any order
Arrays.stream(staff)
    .parallel()
    .forEach(System.out::println); // mixed order

// Need encounter order? use forEachOrdered
Arrays.stream(staff)
    .parallel()
    .forEachOrdered(System.out::println);

// Cost: forEachOrdered may reduce parallel speedup
```

When Parallel Helps — and When Not

Often useful

- Large arrays (thousands+)
- Heavy per-element work
- Independent ops
(map, filter, sum)
- Stateless lambdas

Often not worth it

- Small Employee[]
- Tiny/simple pay()
- Need strict order
- Shared mutable state
in the lambda (unsafe)

Safety — Avoid Shared Mutable State

```
// X Dangerous with parallel  
double[] box = { 0 };  
Arrays.stream(staff).parallel()  
    .forEach(e -> box[0] += e.pay()); // race!
```

```
// ✓ Prefer reduction  
double total = Arrays.stream(staff).parallel()  
    .mapToDouble(Employee::pay)  
    .sum(); // thread-safe reduction
```

Key Takeaways — Parallel Stream

- 01 `.parallel()` runs the pipeline on multiple threads
- 02 Best for large data and independent, heavy work
- 03 `forEach` order is undefined — use `forEachOrdered` if needed
- 04 Do not mutate shared variables inside the lambda
- 05 `sum` / `reduce` are the safe way to aggregate in parallel

Chapter Summary — One-Dimensional Arrays

01 Primitive arrays

declare · length · index · defaults · for / for-each

02 Operations

search · sum · min/max · insert/delete with shift · sort

03 Object arrays

Employee[] · references · null · encapsulation in Manager

04 CRUD · sort · filter

add/find/update/remove · Comparator · Arrays utility

05 Stream processing

lambda · stream · parallel stream on arrays

06 Looking ahead

size vs length · dynamic array · Collections next